OpCode-Stats: A Statistical Analysis of Compiled Program Instruction Sequences

Ara Aslyan
Strike Technologies, New York, USA
aaslyan@striketechnologies.com

Ashot Harutyunyan
Yerevan State University, Machine Learning Lab,
and Institute for Informatics and Automation Problems, Armenia
harutyunyan.ashot@ysu.am

***Abstract*—Compiled binaries, viewed as flat sequences of machine opcodes, exhibit statistical structure analogous to natural language and biological sequences. We present *OpCode-Stats*[1], an open-source toolkit that applies information-theoretic and computational biology techniques to instruction streams. Across multiple corpora (15–40 binaries, 2.9–3.7 M instructions), three independent validation sets, and a 100-binary compiler matrix, we measure opcode frequency distributions, n-gram entropy rates, compressibility, motifs, and manifold dimensionality. Programs occupy a thin structured manifold (d_{95}=8 of 200 bigram dimensions) with strongly Zipfian opcode concentration ($\hat{\alpha}_{\mathrm{MLE}}\approx 1.4$); a bigram language model attains perplexity 8.5, within 0.07 bits of the empirical conditional-entropy lower bound. Systematic ablation shows the structure persists after aggressive boilerplate stripping (max 3.2% change), replicates across corpora, and is compiler-invariant (compiler $R^2<0.03$).**

***Index Terms*—binary analysis, opcode statistics, information theory, language models, compression, Zipf's law**

I. Introduction

Source code is parsed, compiled through many transformations, and reduced to a sequence of machine opcodes that discards all high-level structure. Yet, as we show, this opcode stream retains profound statistical regularity. Bioinformatics offers a natural toolkit for such sequences: k-mer analysis, entropy profiling, motif discovery, and information-theoretic metrics [1]. We extend this toolkit to compiled binaries.

Contributions. (i) An open-source analysis toolkit, *OpCode-Stats*, applying computational-biology techniques to compiled binaries. (ii) Empirical characterisation of opcode frequency concentration: heavy-tailed rank-frequency distribution (MLE $\hat{\alpha}\approx 1.4$; KS 0.117); a unigram–sequential decomposition shows 69% of compression gain over random arises from frequency skew, 31% from sequential structure. (iii) Quantification of n-gram entropy and deviation from a unigram-shuffled baseline (gap 1.12 bits at n=5), revealing multi-scale dependencies. (iv) A controlled compiler matrix (5 projects × {gcc,clang} × {-O0,-O2,-O3}) showing source identity and compiler identity are comparably strong determinants of similarity (both $p<10^{-4}$, $r\approx 0.5$; project vs. compiler: p=0.38); compiler variation adds new manifold dimensions (d_{95} doubles from 8 to 17). (v) Empirical validation that n-gram language model perplexity tracks the conditional-entropy lower bound (bigram gap <0.07 bits).

[1] https://github.com/OpCode-Stats/OpCode-Stats

II. Background and Related Work

Opcode-level analysis. Bilar [2] observed Zipf-like distributions in PE files; subsequent work used n-gram opcode features for malware classification [3], [4], focused on discriminative accuracy rather than information structure. The closest prior cross-domain study is Gan et al. [5], who applied n-tuple Zipf analysis to DNA, English, source, and binary. We extend with mutual-information decay, LZ78 complexity, motif discovery, clone detection, and a controlled compiler experiment. In the malware domain, MAlign [6] encodes raw bytes as nucleotides for genome-style alignment. Recent binary embedding systems (PalmTree [7], jTrans [8], HermesSim [9], Nova [10]) implicitly rely on the regularities OpCode-Stats makes explicit; Marcelli et al. [11] provide a benchmark for similarity search.

Compression and information theory. NCD [12] provides a parameter-free similarity measure rooted in Kolmogorov complexity. The *naturalness* hypothesis [13], [14] states software is more predictable than natural language; cross-entropy of token-level LMs on code is lower than on prose. We extend this to the binary level. Motif discovery from genomics [15] has analogues in compiler-idiom recognition [16], but systematic statistical characterisation is absent from prior literature.

III. Methodology

Corpora. We analyse two corpora of GNU/Linux ELF x86-64 binaries from `/usr/bin`: *Smoke* (15 binaries: grep, sed, gzip, bzip2, xz, find, tar, bash, python3, curl, git, vim, nano, gcc, objdump) and *Coreutils-system* (40 binaries spanning 11 functional categories, $\leq$ 10 MB each). Sequences are extracted via `objdump -d` (60 s timeout per binary); only mnemonics are retained, yielding $s=(o_1,\ldots,o_N)$ over a vocabulary $\mathcal{V}$.

Frequency and Zipf. We fit Zipf's law $f(v)\propto r(v)^{-\alpha}$ via OLS on log-log coordinates and via MLE on the truncated discrete Zipf model ($\ell(\alpha)=-\alpha\sum_r f(r)\log r - N\log Z_V(\alpha)$ with $Z_V(\alpha)=\sum_{r=1}^{V} r^{-\alpha}$). 95% CIs come from n=200 bootstrap resamples; goodness-of-fit is reported via the Kolmogorov–Smirnov statistic against the fitted rank-CDF.

Entropy and shuffled baseline. n-gram entropy $H_n=-\sum_w p(w)\log_2 p(w)$ and entropy rate $h_r(n)=H_n/n$ are computed for $n=1,\ldots,5$. A unigram-preserving shuffle baseline isolates higher-order dependencies; a negative *entropy gain* $\Delta h_r(n)=h_r(n)^{\mathrm{real}}-h_r(n)^{\mathrm{shuf}}$ for $n>1$ confirms genuine multi-opcode structure.

TABLE I
CORPUS STATISTICS AFTER INODE DEDUPLICATION.

Corpus	Bins	Cats	Vocab	Instr. (M)
Smoke	15	8	281	2.91
Coreutils-sys	40	11	301	3.74

Compression and LZ78. We measure compression ratios $\rho{=}|C(s)|/|s|$ under `zlib` and `lzma`, and LZ78 normalised complexity $\tilde{c}{=}K/\log_2 N$ [17] (where K is the distinct-phrase count). Both are compared to a uniform-random baseline (maximum entropy) and a unigram-shuffled baseline (preserves marginals). The gap between real and shuffled isolates compression gain attributable to *sequential* structure beyond Zipfian skew.

Motifs, similarity, manifold. A k-mer motif is a tuple $(o_i, \ldots, o_{i+k-1})$; we enumerate motifs for $k{\in}\{4, \ldots, 12\}$ and retain those covering $\geq 1\%$ of functions. Pairwise similarity uses NCD $(C(xy) - \min(C(x), C(y)))/\max(C(x), C(y))$ with `zlib`/`lzma`, and L1-normalised n-gram cosine. Mutual information $I(o_t; o_{t+\ell})$ is estimated for lags $\ell{=}1, \ldots, 50$. Corpus manifold dimensionality d_{95} is the PCA component count explaining 95% of inter-binary variance on top-200 bigram frequency vectors.

n-gram language models. We train Laplace-smoothed n-gram LMs ($n{=}1, \ldots, 5$) jointly on an 80/20 token split. We report cross-entropy $H_{\mathrm{LM}}(n){=}-\frac{1}{T}\sum_t \log_2 P(o_t|o_{t-n+1:t-1})$, perplexity $\mathrm{PPL}(n){=}2^{H_{\mathrm{LM}}(n)}$, and the smoothing gap $\delta(n){=}H_{\mathrm{LM}}(n) - (H_n - H_{n-1})$ vs. the empirical conditional-entropy lower bound.

Clone detection. A three-stage pipeline: MinHash signatures ($k{\in}\{3, 4, 5\}$, $m{=}128$ permutations), LSH at Jaccard thresholds $\tau{\in}\{0.70, 0.90\}$, and Smith–Waterman alignment with normalised score $s{=}S/\min(|f_1|, |f_2|)$. Pairs are classified Type 1 ($J{\geq}0.95$), Type 2 ($J{\geq}0.70$), Type 3 ($J{\geq}0.50$); families are connected components.

IV. RESULTS

A. Corpus and Vocabulary

After inode deduplication, the smoke corpus contains 15 binaries (2.91 M instructions, $|\mathcal{V}|{=}281$); coreutils-system contains 40 binaries (3.74 M instructions, $|\mathcal{V}|{=}301$), spanning 11 functional categories (Table I).

The top-5 opcodes (`mov`, `lea`, `call`, `jmp`, `push`) account for over 50% of instructions.

Zipf's law. OLS on log-log coordinates yields $\hat{\alpha}_{\mathrm{OLS}}{=}3.34$ (95% CI $[3.05, 3.67]$, $R^2{=}0.92$). MLE on the truncated discrete Zipf model yields $\hat{\alpha}_{\mathrm{MLE}}{=}1.44$ (95% CI $[1.438, 1.440]$, KS 0.117). The two diverge because OLS overweights the long tail of near-zero-frequency opcodes; MLE, properly weighted by counts, lies closer to natural-language $\alpha{\approx}1$. The KS statistic exceeds the asymptotic critical value at $n{=}281$ (0.081), so a strict truncated power law is rejected; the distribution is best characterised as heavy-tailed with approximately Zipfian concentration (Fig. 1).

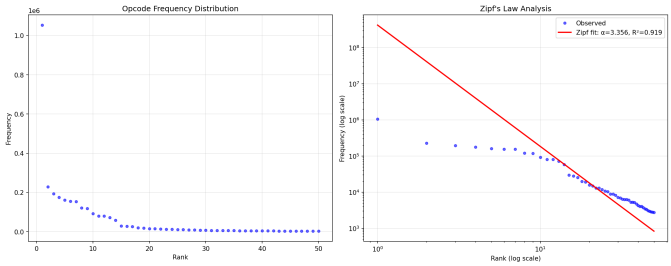

Fig. 1. Opcode rank–frequency distribution (log-log, smoke corpus). The red line is the Zipf fit; the shaded band is the 95% bootstrap CI on α. A handful of mnemonics dominate; the tail decays steeply.

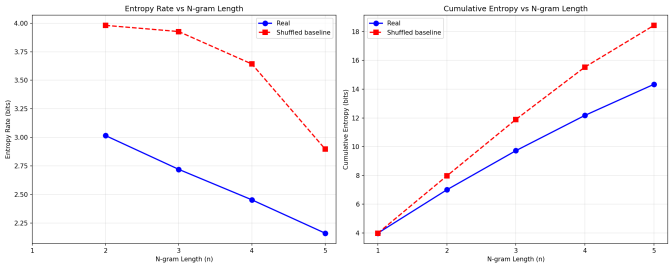

Fig. 2. Entropy rate $h_r(n)$ vs. n for real (blue) and unigram-shuffled (red dashed). The widening gap evidences multi-scale sequential dependencies.

B. n-gram Entropy

The entropy rate of real sequences decreases by 1.12 bits from $n{=}1$ ($h_r(1){=}3.98$) to $n{=}5$ ($h_r(5){=}2.86$); the shuffled baseline decreases by only 0.30 bits, reflecting near-independence after shuffling. $\Delta h_r(n){<}0$ for all $n{>}1$ confirms genuine multi-instruction dependencies (Fig. 2). Per-binary analysis shows shells and compilers exhibit lower entropy than compression utilities, consistent with predictable control-flow-heavy code vs. data-transform pipelines.

C. Compression and LZ78

Real binaries compress to $\rho_{\texttt{zlib}}{=}0.221$ vs. 0.360 shuffled and 0.666 random; `lzma` gives 0.184/0.303/0.553 (Table II). Decomposing $\Delta{=}0.445$ over random, unigram skew accounts for 0.306 (69%) and sequential order for 0.139 (31%). LZ78 normalised complexity is 630 (median, real) vs. 45,187 (shuffled) and 87,881 (random)—the relative separation, not absolute scale, is the diagnostic since $\tilde{c}$ scales with $N/\log^2 N$.

D. Motifs and Function Boundaries

Motif discovery across $k{=}4{-}12$ identifies 7,623 unique patterns meeting frequency/coverage thresholds; the top-100 per length yield 798 motifs. High-scoring motifs fall into three categories: function epilogues (top-ranked 4-mer `pop pop pop ret` appears in 27.9% of functions), call sequences (`call` preceded by argument-loading `lea`/`mov`), and prologues (`endbr64; push; mov`, reflecting CET + ABI conventions). Position-0 of functions is dominated by `endbr64` ($p{\approx}0.986$, system-dependent on CET); positions 1–2 are `push` (0.672) and `mov` (0.671), ABI-mandated. Boundary entropy at position 0 is 0.12 bits (vs. corpus unigram entropy

TABLE II
COMPRESSION $\rho = |C(s)|/|s|$ (LOWER IS MORE COMPRESSIBLE).

	Real	Shuffled	Random
`zlib`	0.221	0.360	0.666
`lzma`	0.184	0.303	0.553
LZ78 (norm.)	630 (med.)	45,187	87,881

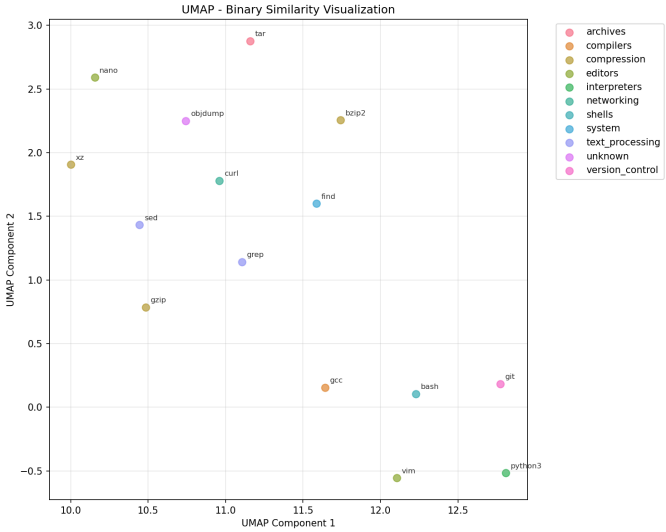

Fig. 3. UMAP projection of the NCD-`zlib` distance matrix (smoke corpus); points coloured by functional category. Compression utilities cluster apart from large interactive programs.

3.99); 3 conserved start positions and 0 conserved end positions classify the corpus as loosely structured.

E. Similarity and Manifold

The smoke-corpus NCD matrix (`zlib`) ranges 0.956–1.000 (mean 0.991); under `lzma` it ranges 0.852–0.993 (mean 0.967). Closest pairs are intra-category compression utilities (`bzip2`–`xz`: 0.956). A one-sided Mann–Whitney U test confirms within-category pairs are stochastically closer than between (`zlib`: p=0.013, permutation p<0.001, r=0.600; `lzma`: p=0.005, permutation p=0.003, r=0.698). The absolute gap is small (mean within 0.972 vs. between 0.992 under `zlib`: 13.8% of the between-group baseline), so we interpret NCD as a weak but significant continuous signal, not a basis for hard clustering.

N-gram cosine similarity over L1-normalised TF vectors decreases with n: 0.953 (bigrams), 0.897 (trigrams), 0.812 (4-grams), with standard deviations 0.037/0.080/0.132. The growing spread is consistent with rare n-grams becoming discriminative as n grows. The UMAP projection of the NCD-`zlib` matrix (Fig. 3) places small compression utilities in a region distinct from large interactive programs (`bash`, `vim`, `git`, `python3`).

Manifold dimensionality. Even at n=2, only 6.7% of possible opcode pairs are observed (5,757 of 85,849); coverage drops to 0.15% at n=3. N-gram space coverage drops

TABLE III
n-GRAM SPACE COVERAGE: UNIQUE OBSERVED VS. THEORETICAL MAXIMUM (SMOKE CORPUS).

n	Unique	Theoretical max	Coverage
1	281	281	100%
2	5,757	85,849	6.71%
3	38,073	25,153,757	0.15%
4	137,534	7.4×10^9	1.9×10^{-3}%

TABLE IV
n-GRAM LM CROSS-ENTROPY AND PERPLEXITY VS. EMPIRICAL CONDITIONAL-ENTROPY BOUND. LAPLACE SMOOTHING, 80/20 SPLIT.

Model	CE (b)	PPL	Bound (b)	δ (b)
Unigram	4.29	19.6	3.99	+0.30
Bigram	3.08	8.5	3.02	+0.07
Trigram	2.79	6.9	2.72	+0.07
4-gram	2.84	7.1	2.45	+0.38
5-gram	3.13	8.8	2.16	+0.97

sharply with n (Table III): even bigrams cover only 6.7% of the theoretical maximum; trigrams 0.15%; 4-grams $\sim10^{-5}$. Corpus-level PCA on top-200 bigram vectors finds d_{95}=8 of 200 (4%)—a thin structured manifold.

A Levina–Bickel MLE [18] cross-check gives $\hat{d}_{\mathrm{MLE}}=8.0\pm5.8$.

Mutual information decay. $I(1)\approx1.12$ bits, far above shuffled 0.086; MI half-life is $\ell^*\approx2$, indicating predominantly local structure. At lag 50, $I(50)$=0.232 vs. shuffled 0.086, suggesting weak but non-zero long-range dependencies consistent with repetitive call/ret patterns.

F. n-gram Language Model Perplexity

Table IV reports the LM evaluation. The unigram LM achieves PPL 19.6 despite a vocabulary of 281—a 15× effective reduction. The bigram LM reaches PPL 8.5 with smoothing gap $\delta(2)$=0.07 bits, within 2% of the conditional-entropy bound; trigram is similarly tight ($\delta(3)$=0.07). For $n\geq4$ the gap grows ($\delta(4)$=0.38, $\delta(5)$=0.97) as Laplace overhead dominates the sparse high-order context space. The tight n=2, 3 match validates the information-theoretic analysis: corpus-derived entropy rates accurately predict trained-LM perplexity (Fig. 4).

G. Compiler Matrix

We compile five C programs (`sort`, `hash`, `compress`, `search`, `matrix`) under {`gcc`, `clang`} × {`-O0`, `-O2`, `-O3`}, producing 30 binaries. Per-optimisation d_{95}=8 at every level (10 binaries each); the full 30-binary corpus has d_{95}=17, showing compiler/optimisation choice adds new manifold dimensions beyond the source-program span. $-O3$ increases mean instruction count (601 vs. 499 at $-O0$) but also compressibility for compute-intensive code (`matrix` achieves ρ=0.375 via loop unrolling [19]); $-O2$ produces the *least*-compressible output (Table V).

For all $\binom{30}{2}$=435 pairs, one-sided Mann–Whitney tests give within-project $p<10^{-4}$ (r=0.51), within-compiler $p<10^{-4}$

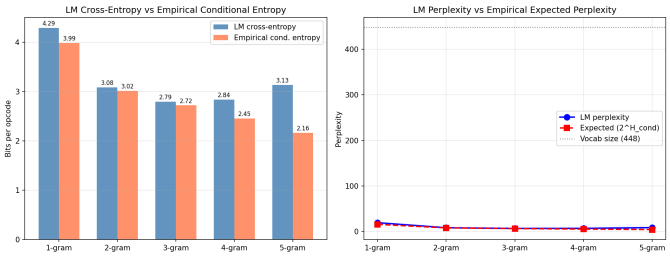

Fig. 4. LM perplexity (blue) vs. expected perplexity $2^{H_n - H_{n-1}}$ (red dashed) and vocabulary size (grey dotted) by n-gram order. Bigram and trigram LMs lie within 0.07 bits of the theoretical lower bound; Laplace overhead dominates at $n \geq 4$.

TABLE V
MEAN INSTRUCTION COUNT AND ZLIB RATIO BY OPTIMISATION LEVEL (AVERAGED OVER 5 PROJECTS × 2 COMPILERS = 10 BINARIES PER ROW).

Opt	Mean instr.	Mean `zlib`
`-O0`	499	0.458
`-O2`	434	0.548
`-O3`	601	0.480

(r=0.55); within-project is *not* significantly tighter than within-compiler (p=0.38, Table VI). Source identity and compiler identity are comparably strong determinants of similarity.

H. Clone Detection

LSH yielded 556 unique candidate pairs on the 6,025 functions of the smoke corpus; Smith–Waterman alignment confirmed 537 (96.6%). These form 117 families, four of which span multiple binaries. Type 1 (exact, $J \geq 0.95$) dominates (63.7%), consistent with verbatim compiler/linker boilerplate (function prologues, `__libc_start_main` wrappers). 5.8% of all functions are in some clone family; the largest family (size 14) is the standard ABI prologue. Stub-exclusion analysis (below) shows 46% of functions are toolchain-generated, so cross-binary clones include both toolchain-inserted and programmer-authored patterns—the latter a smaller subset. Per-binary clone density varies from 9.6% (`bash`, reflecting its inline library of shell built-in helpers) and 5.0% (`sed`) down to <0.5% for `git` and `vim`, suggesting hand-crafted code with unique control flow is less prone to accidental near-duplication.

V. Validation and Robustness

We conducted a systematic validation programme targeting four threats: extraction errors, boilerplate confounds, corpus dependence, and compiler dependence; full scripts and intermediate data are in `validation/`.

Extraction. An independent reference parser was applied to five binaries (`grep`, `bash`, `find`, `tar`, `xz`); per-mnemonic counts matched exactly (e.g. `bash`: 241,789 instructions). Hard-link deduplication was verified against 7 inode-sharing groups; timeout handling was confirmed via mock injection.

Synthetic and bootstrap checks. Five synthetic corpus families (uniform, Zipf-shuffled, order-2 Markov, templated, mixed) passed all 26 metric checks. Binary-level bootstrap (N=200) gives MLE $\hat{\alpha}$=1.44 [1.36, 1.53], $h_r(5)$=2.86 [2.79, 2.94] bits, mean `zlib` ratio 0.22 [0.18, 0.27]; all CIs exclude reversal. Leave-one-binary-out identifies no influential outlier ($\hat{\alpha}$ varies ≤2%).

TABLE VI
LZMA NCD BY PAIR TYPE ON THE 30-BINARY COMPILER MATRIX. ONE-SIDED MANN–WHITNEY U VS. "BETWEEN".

Pair type	n	NCD	U	p	r
Within-project	75	0.496	5,537	$< 10^{-4}$	0.51
Within-compiler	60	0.539	4,053	$< 10^{-4}$	0.55
Between	300	0.576	—	—	—

TABLE VII
BOILERPLATE ABLATION (SMOKE CORPUS). ALL VARIANTS PRESERVE THE ENTROPY GAP AND ZIPF CONCENTRATION.

Variant	$\hat{\alpha}$	$h_r(5)$	Gap	`zlib`
Original	1.439	2.863	−0.822	0.221
Strip first 5	1.442	2.867	−0.810	0.224
Strip last 5	1.442	2.866	−0.811	0.224
Strip both 5	1.443	2.866	−0.808	0.224
Remove `endbr64`	1.447	2.862	−0.799	0.225
Full boilerplate strip	1.447	2.865	−0.796	0.225

Boilerplate ablation. Sixteen ablated variants (strip first/last $k \in \{1, 2, 3, 5\}$ instructions, remove `endbr64`, strip prologues/epilogues, full strip) preserve all metrics within 3.2% (Table VII). Toolchain functions (startup + stubs) account for 46% of functions but only 0.6% of instructions; removing them changes mean `zlib` by −0.006 and $h_r(5)$ by +0.003. The findings are intrinsic to application code, not boilerplate. Operand-aware variants (opcode+register class, opcode+immediate bucket) confirm the entropy gap persists.

External validation. Three disjoint corpora pass all pre-registered consistency criteria (Table IX): real entropy below shuffled, real compression below shuffled, cross-corpus bigram LM perplexity within 1.2× of within-corpus. An expanded matrix of 100 binaries ({`gcc`, `clang`} × {`-O0`, `-O1`, `-O2`, `-O3`, `-Os`} × 10 programs) gives one-way ANOVA R^2: program identity dominates (R^2=0.40−0.66), compiler identity <3% for all four metrics (Zipf α, H_1, $h_r(2)$, `zlib` ratio), confirming compiler-invariance. Scaling from the 15-binary smoke corpus to the 40-binary coreutils-system superset (Table VIII) shows MLE $\hat{\alpha}$, entropy rates, and compression ratios are nearly identical, with manifold dimensionality growing modestly (d_{95}:8→14) as expected when category breadth increases.

Validation summary. Of ten main empirical claims, eight are rated **strong** (CI excludes reversal, ablation < 3.2%, replicates across corpora, compiler-invariant), one **moderate** (cross-binary clones are likely dominated by PLT stubs), one **unsupported** (categorical clustering by binary type: NCD within-vs-between gap is only 13.8% of the between-group baseline).

VI. Discussion

Compiled programs do not explore opcode-sequence space uniformly. They are confined to a low-dimensional, highly

TABLE VIII
METRIC STABILITY ACROSS CORPUS SCALE (SMOKE IS A SUBSET OF COREUTILS-SYSTEM).

Metric	Smoke (15)	Coreutils (40)
Instructions (M)	2.91	3.74
Vocabulary $\|\mathcal{V}\|$	281	301
MLE $\hat{\alpha}$	1.439	1.441
$h_r(1)$ (bits)	3.98	3.99
$h_r(5)$ (bits)	2.86	2.88
`zlib` ratio (real)	0.221	0.237
PCA d_{95} (bigram)	8/200	14/200

TABLE IX
METRIC CONSISTENCY ACROSS THREE INDEPENDENT CORPORA.

Corpus	N	H_1	$h_r(5)$	Gap	`zlib`
A (smoke)	15	3.98	2.86	-0.14	0.221
B (net/build)	15	3.97	2.77	-0.21	0.369
C (dev tools)	9	3.99	2.79	-0.17	0.235

structured manifold shaped by ABI conventions (CET + frame setup), compiler optimisation idioms, algorithmic patterns, and clone reuse (linker-inserted stubs and shared libc helpers). Concentrated vocabulary (MLE $\hat{\alpha}\approx 1.4$), rapidly decaying entropy rates, very high compressibility relative to baselines, and $d_{95}\approx 8$ collectively show programs are extraordinarily predictable at the instruction level. The validation results (Section V) demonstrate this is not a boilerplate artefact: stripping changes metrics by $\leq 3.2\%$, findings replicate across three corpora, and compiler identity explains $<3\%$ of variance.

Implications for LMs on code. The naturalness hypothesis [13] attributes LLM success on source code to its statistical regularity. Our results extend this to the binary level: an LM faces a small effective vocabulary, highly non-uniform transitions, MI decay within ≈ 2 steps, and low intrinsic dimensionality. The bigram LM (Section IV-F) achieves PPL 8.5 within 0.07 bits of the conditional-entropy bound—simple count statistics already capture nearly all bigram-order structure.

Threats to validity. Construct: compression conflates frequency skew with sequential redundancy, mitigated but not eliminated by the shuffled baseline. Internal: the boilerplate-driven hypothesis is rejected (Section V); operand-aware variants confirm the entropy gap. External: all binaries are x86-64 ELF on GNU/Linux; ARM64, RISC-V, WASM, MSVC, and other source languages are untested. Statistical: $N=15$ is a convenience sample; tokens within a binary are non-independent, so corpus-level claims effectively have $N=15$ observations, not millions.

VII. CONCLUSION

Across a 15-binary smoke corpus (2.91 M instructions), three independent validation corpora, and a 100-binary compiler matrix, opcode frequencies are heavily concentrated (MLE $\hat{\alpha}\approx 1.4$), entropy rates decrease by 1.12 bits from $n=1$ to $n=5$ vs. 0.30 shuffled, real sequences compress to 22% vs. 36% shuffled and 67% random, 7,623 unique motifs capture compiler idioms, the manifold occupies $d_{95}=8$ of 200 bigram dimensions, and a bigram LM attains PPL 8.5 within 0.07 bits of the lower bound. Eight of ten empirical claims are strong, one moderate, one unsupported. The thin-manifold hypothesis explains why language models and similarity search are effective on binary code.

Future work. (i) multi-ISA replication (ARM64, RISC-V, WASM, JVM bytecode); (ii) operand-level entropy and manifold analysis; (iii) compiler / source-language fingerprinting; (iv) Transformer-LM evaluation vs. the n-gram baseline; (v) clone-guided vulnerability propagation across binaries sharing toolchain or application clones.

ACKNOWLEDGEMENTS

Portions of the writing and code in this work were assisted by large-language-model tools (OpenAI ChatGPT, Anthropic Claude); all experimental design, results, and analysis were verified by the authors.

REFERENCES

[1] S. Vinga and J. Almeida, "Alignment-free sequence comparison – a review," *Bioinformatics*, vol. 19, no. 4, pp. 513–523, 2003.

[2] D. Bilar, "Opcodes as predictor for malware," *International Journal of Electronic Security and Digital Forensics*, vol. 1, no. 2, pp. 156–168, 2007.

[3] M. M. Masud, L. Khan, and B. Thuraisingham, "A scalable multi-level feature extraction technique to detect malicious executables," in *Policy and Models in System and Network Security*, 2008.

[4] I. Santos, Y. K. Penya, J. Devesa, and P. G. Bringas, "N-grams-based file signatures for malware detection," *ICEIS*, vol. 2, pp. 317–320, 2009.

[5] Q. Gan, J. Wang, and J. Han, "Statistical analysis of n-tuple patterns in DNA, natural language, source code, and binary code," in *IEEE International Conference on Data Mining (ICDM) Workshops*, 2009, pp. 183–188.

[6] S. Saha, S. Bhatt, and S. Mehnaz, "MAlign: Towards better malware family clustering via sequence alignment," *Computers & Security*, vol. 140, p. 103748, 2024.

[7] X. Li, Y. Qu, and H. Yin, "PalmTree: Learning an assembly language model for instruction embedding," in *ACM Conference on Computer and Communications Security (CCS)*, 2021, pp. 3236–3251.

[8] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "jTrans: Jump-aware transformer for binary code similarity detection," in *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022, pp. 1–13.

[9] H. Xu *et al.*, "HermesSim: Semantics-oriented binary code similarity analysis with efficient multi-task learning," in *USENIX Security Symposium*, 2024.

[10] N. Jiang, S. Huang *et al.*, "Nova: Generative language models for assembly code with hierarchical attention and contrastive learning," in *International Conference on Learning Representations (ICLR)*, 2025.

[11] A. Marcelli, M. Graziano, X. Ugarte-Pedrero, Y. Fratantonio, M. Mansouri, and D. Balzarotti, "How machine learning is solving the binary function similarity problem," in *USENIX Security Symposium*, 2022, pp. 2099–2116.

[12] R. Cilibrasi and P. M. B. Vitányi, "Clustering by compression," *IEEE Transactions on Information Theory*, vol. 51, no. 4, pp. 1523–1545, 2005.

[13] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 837–847.

[14] Z. Tu, Z. Su, and P. Devanbu, "On the localness of software," in *Foundations of Software Engineering (FSE)*. ACM, 2014, pp. 269–280.

[15] M. K. Das and H.-K. Dai, "A survey of DNA motif finding algorithms," in *BMC Bioinformatics*, vol. 8, no. S7, 2007, p. S21.

[16] E. Schulte, J. Ruchti, M. Noonan, D. Ciarletta, and A. Loginov, "Evolving exact decompilation," in *Binary Analysis Research (BAR) Workshop*, 2018.

[17] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Transactions on Information Theory*, vol. 24, no. 5, pp. 530–536, 1978.

[18] E. Levina and P. J. Bickel, "Maximum likelihood estimation of intrinsic dimension," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 17, 2005.

[19] A. Fog, *Optimizing Software in C++: An Optimization Guide for Windows, Linux, and Mac Platforms*, 2024, available at https://agner.org/optimize/.